week03

named parameter

- 순서와 상관없이 지정하고 싶은 매개변수의 이름을 이용해 값을 입력

```
int addTwoNumbers({
required int a;
//required는 매개변수가 null값이 불가능한 타입이면 기본값 지정, 필수로 입력해야 한다는 의미
required int b;
}){
return a+b;
}

void main(){
print(addTwoNumbers(a: 1, b: 2));
}
```

출력값: 3

익명 함수와 람다 함수

- 둘다 함수 이름이 없고 일회성으로 사용된다
- 통상적으로 많은 언어에서 익명 함수와 람다 함수를 구분하지만 다트는 구분하지 않는다

```
//익명함수
(매개변수){
    함수 바디
    }

//람다 함수
(매개변수) => 단 하나의 스테이트먼트 //한 줄이 아니라 명령 단위가 하나여야 한다
```

- 가독성이 좋다는 장점이 있다

예시)

```
void main(){
List<int> numbers = [1,2,3,4,5];

//람다 함수로 모든 값 더하기
final allMembers = numbers.reduce((value, element)=>value+e
lement);

print(allMembers);
}
```

출력값: 15

typedef와 함수

- typedef: 함수의 시그니처를 정의한다→ 함수가 무슨 동작을 하는 지에 대한 정의는 없다

시그니처: 반환값 타입, 매개변수의 개수와 타입

- 다트에서 함수는 일급 객체이기에 함수를 값처럼 이용할 수 있다

```
typedef Operation = void Function(int x, int y);

void add(int x, int y){
print('결괏값: ${x+y}');
}

void calculate(int x, int y, Operation oper){
oper(x,y);
}

void main(){
Operation oper1 = add;
oper(1,2);
```

```
calculate(1,2,add);
}
```

try...catch

- 특정 코드의 실행을 시도해보고 문제가 있다면 에러를 잡으라는 뜻

```
void main(){
try{
final String name='코드팩토리';
//에러 없을 때 코드팩토리 출력
throw Exception('이름이 잘못됐습니다!');
//throw 키워드로 고의적으로 에러 발생시킴
print(name);
}catch(e){
print(e);
//에러 있을 때 실행할 로직
//try에서 에러가 났으니 catch 로직이 실행됨
}
}
```